

“Single-Threaded JavaScript

JavaScript runs on **ONE** main thread → only **one** task executes at a time.

JS cannot do two things simultaneously.

It can only **SWITCH** between tasks very fast.

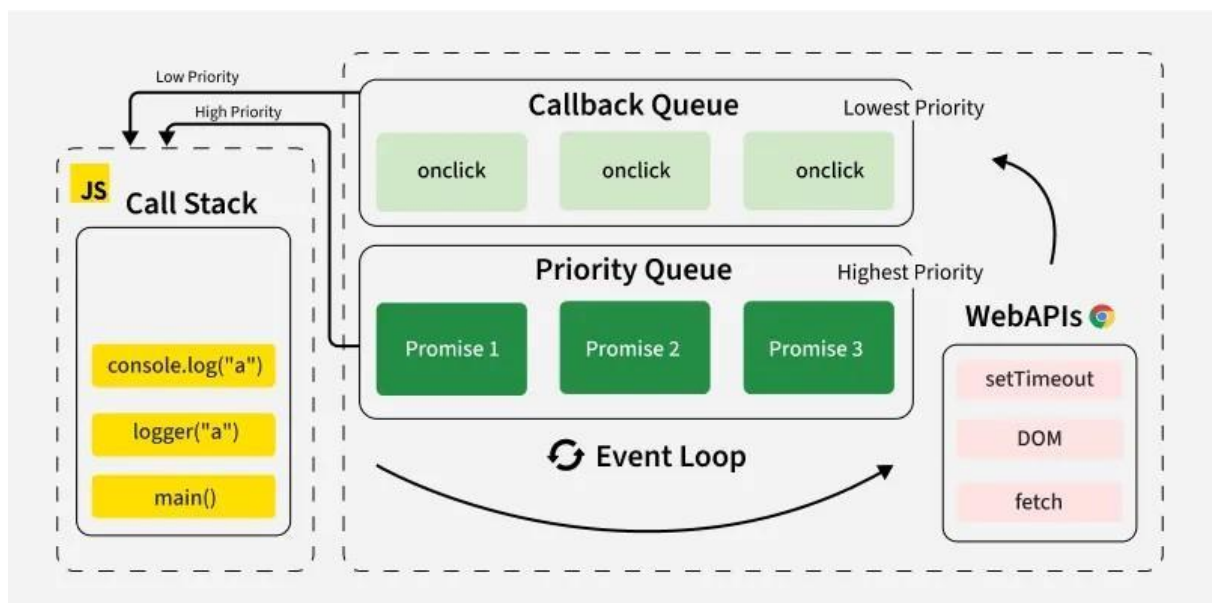
But here's the twist:

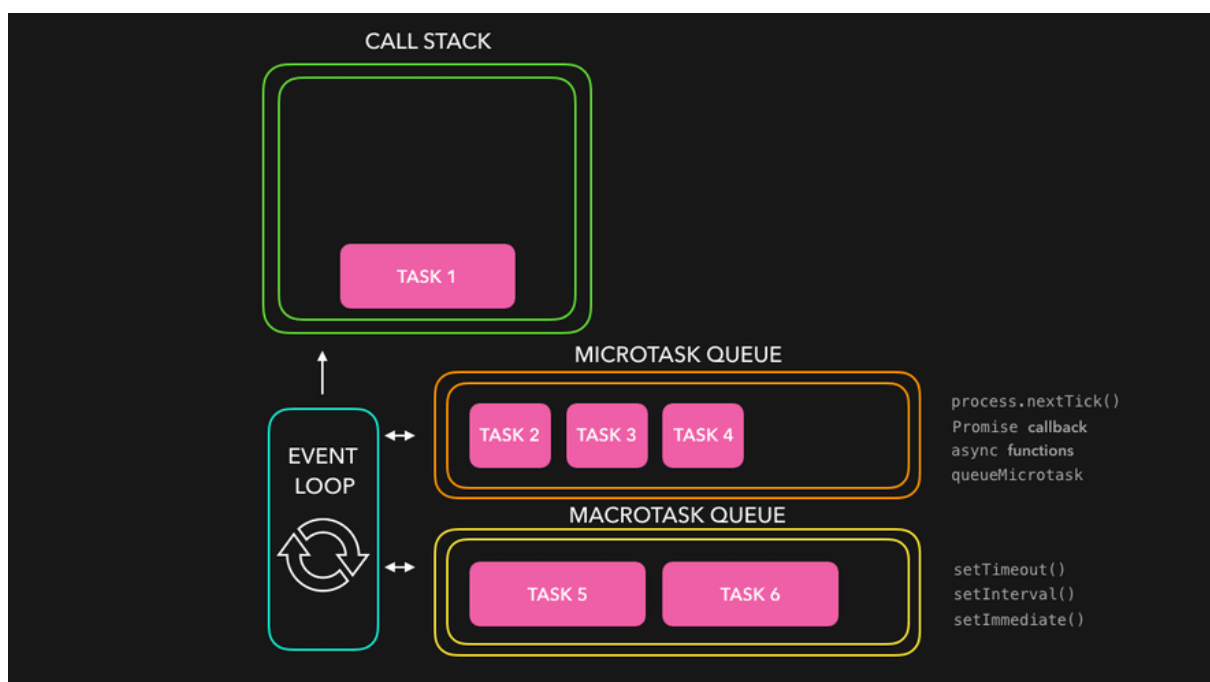
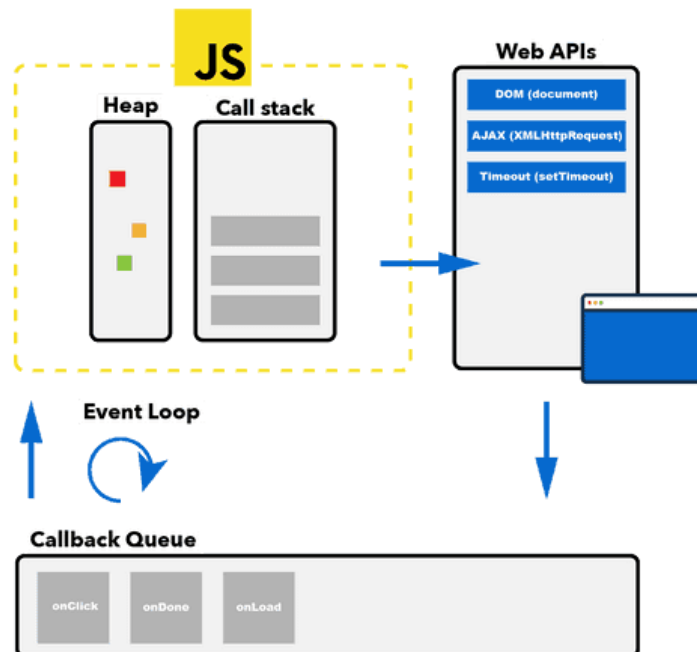
- JS itself is single-threaded
- The environment (Browser / Node.js) is **NOT**.

The environment gives JS superpowers using:

- Web APIs (browser)
- libuv thread pool (Node.js)
- Event Queue
- Event Loop

EVENT LOOP





Component	Role
Call Stack	Executes JS code (only 1 thing allowed)
Web APIs	Handles async tasks (timer, fetch, DOM)
Callback Queue	Stores completed async callbacks
Microtask Queue	Stores Promises (higher priority)
Event Loop	Moves tasks into stack when free

The Call Stack (Execution Engine)

This is where synchronous code runs.

```
function a() {  
  b();  
}  
function b() {  
  console.log("Hello");  
}  
a();
```

push a()

push b()

run console.log

pop b()

pop a()

Stack must be EMPTY before async callback runs.

Why JS Needed Async Model (The Problem)

```
setTimeout(() => console.log("Done"), 5000);
```

If JS waited 5 sec on the stack:

- ✗ UI freezes
- ✗ App becomes unusable
- ✗ No scalability

So JS offloads waiting work to the environment.

Web APIs — The Hidden Multithreading

```
setTimeout(fn, 2000);
```

```
fetch("/data");
```

```
addEventListener("click", fn);
```

It says:

"Hey Browser, YOU handle this. I'll continue."

Browser runs these using:

- Timer threads
- Network threads
- DOM threads

JS doesn't wait.

Callback Queue (Macrotask Queue)

When async finishes:

Timer done → callback goes to QUEUE

But it does NOT run immediately.

It must WAIT until:

Call Stack === Empty

Event Loop — The Brain

The Event Loop continuously checks:

Is stack empty?

Yes → Move next task from queue into stack.

This loop runs millions of times per second.

That's why it's called:

Event Loop = Scheduler of JavaScript

Microtasks vs Macrotasks (The Most Important Interview Concept)

JS has TWO queues, not one.

Type	Examples	Priority
Microtask Queue	Promise.then, queueMicrotask	 HIGH

Type	Examples	Priority
Macrotask Queue	setTimeout, setInterval, I/O	LOW

Execution Priority (Golden Rule)

- 1 Run all Sync Code
- 2 Run ALL Microtasks
- 3 Run ONE Macrotask
- 4 Repeat

Microtasks ALWAYS finish before macrotasks.

Example That Confuses Everyone

```
console.log("Start");
```

```
setTimeout(() => console.log("Timeout"), 0);
```

```
Promise.resolve().then(() => console.log("Promise"));
```

```
console.log("End");
```

Output:

Start

End

Promise

Timeout

Why?

Step-by-step:

- 1 Start → sync → runs
- 2 setTimeout → goes to Web API
- 3 Promise.then → goes to Microtask queue
- 4 End → sync → runs
- 5 Stack empty → run ALL microtasks → Promise
- 6 Then macrotask → Timeout

setTimeout(0) Is NOT Instant

This is misunderstood.

`setTimeout(fn, 0);`

Means:

Run AFTER current execution cycle.

Minimum delay ≈ 4ms (browser throttling).

JavaScript is single-threaded, but achieves concurrency through the event loop, task queues, and host-provided asynchronous APIs, allowing non-blocking execution despite having only one execution stack.

Browser vs Node.js — Async Behaviour Difference

When JavaScript runs in a browser:

It says:

“Hey browser, I need to wait 2 seconds / fetch data / listen for a click...
You handle it, I’ll continue working.”

So the browser itself does the waiting and background work.

Example:

- Timer → handled by browser clock
- Fetch → handled by browser network team
- Click → handled by browser UI system

JavaScript just gets called back when ready.

☞ Browser async = “Outsource to the browser.”

In Node.js → The Worker Has a Backend Team (libuv + OS)

Node.js doesn't have a browser.

So it created its own mini backend team (called libuv).

When JS says:

“Read this file” or “Handle this request”

Node sends that job to:

- OS
- Hidden thread pool
- System-level I/O handlers

Then JS continues working until results come back.

☞ Node async = “Outsource to the system.”

🎯 Real-Life Analogy

Browser = Restaurant with Waiters

- JS = Chef (single person cooking)
 - Browser = Waiters taking orders & serving tables
 - Chef keeps cooking without leaving kitchen.
-

Node.js = Cloud Kitchen with Dispatch System

- JS = Head cook
 - libuv/OS = Delivery team + helpers
 - Orders processed in background, cook keeps moving.
-

⚡ Key Difference in One Line

Browser async is designed to keep the UI smooth.

Node.js async is designed to handle thousands of tasks efficiently.

JavaScript language is the same, but the runtime environment is different.

Async behavior depends on who is hosting JS:

- Browser → uses Web APIs
- Node.js → uses libuv + OS

That's why timing, queues, and execution feel slightly different.

Who Handles Async Work?

The Browser provides built-in APIs:

Feature	Handled By
setTimeout	Timer API
fetch	Network API
DOM events	Event system
addEventListener	UI thread
Rendering	Render engine

These APIs run outside the JS engine.

Browser Execution Flow

JS Call Stack → gives async task to Web API

Web API completes work

Callback goes to Task Queue

Event Loop pushes it back when stack is empty

Special Browser Behavior: Rendering Matters

Browser must also manage:

- Layout
- Painting
- Reflow
- FPS (60fps target)

So it schedules tasks like:

JS → Microtasks → Render → Next Macrotask

That's why blocking JS freezes UI.

Node.js Async Model (libuv-Driven)

Node.js is not a browser, so no Web APIs exist.

Instead it uses:

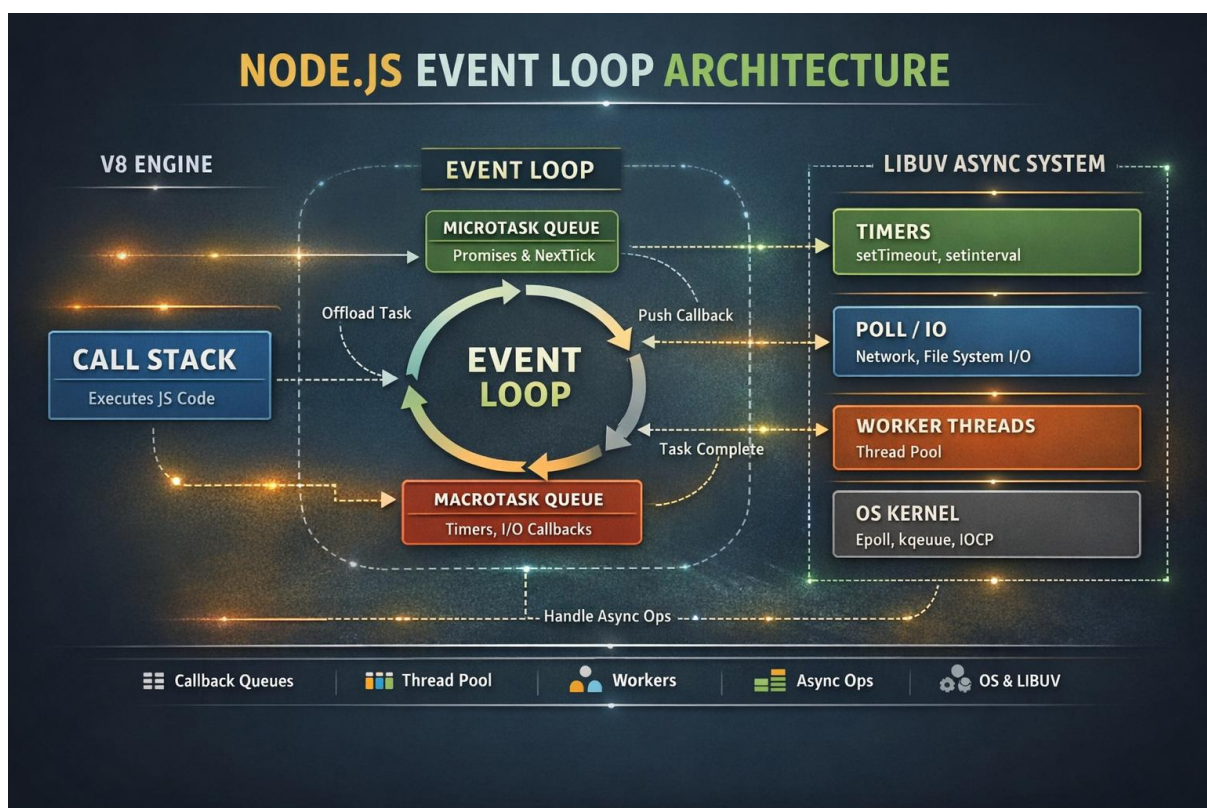
libuv → a C++ library that creates a thread pool + event loop.

Who Handles Async Work?

Feature	Handled By
File system	libuv thread pool
Network	OS kernel (epoll/kqueue)
Timers	libuv timers

Feature	Handled By
Crypto	Worker threads
DNS	Thread pool

Node secretly uses multiple threads, but JS still runs on one.



Node.js Event Loop Has PHASES (Browser Doesn't)

Browser has a simple model:

Microtask Queue → Macrotask Queue

Node.js has 6 internal phases:

- 1** Timers (setTimeout)
- 2** Pending Callbacks
- 3** Idle / Prepare
- 4** Poll (I/O happens here)
- 5** Check (setImmediate)
- 6** Close Callbacks

Microtask Handling Difference (Important!)

Both support:

`Promise.then()`

`queueMicrotask()`

But Node.js runs microtasks:

After EVERY phase

Browser runs microtasks:

After EACH microtask

That subtle difference can change execution order in edge cases.

5. Threading Difference

Runtime	Threads Used	Why
Browser	Multiple internal threads	UI + network + rendering
Node.js	libuv worker pool (4 default)	Server scalability

In Node:

→ executed in thread pool

→ callback returned to main thread

Browser cannot do this because it has no filesystem access.

Browser Async = API Delegation Model

“Let the browser handle things while JS waits.”

Node Async = Evented I/O Model

“Continuously poll the OS and dispatch completed work.”

Browsers implement async behavior using Web APIs tied to UI and networking, while Node.js uses the libuv event loop and thread pool to manage non-blocking I/O. Both expose the same JavaScript primitives (Promises, timers), but their scheduling, threading, and execution phases differ due to their distinct runtime goals—UI responsiveness vs server scalability.

1. What is the Event Loop?

Interview Answer:

The Event Loop is a mechanism that allows JavaScript, which is single-threaded, to handle asynchronous operations by continuously checking the call stack and moving completed async callbacks from the task queues into the stack when it becomes empty.

2. If JavaScript is Single-Threaded, How Does It Handle Async Tasks?

Interview Answer:

JavaScript itself runs on one thread, but async operations are handled by the host environment (browser or Node.js). These environments execute tasks like timers, network requests, or file I/O in the background and notify JavaScript via queues, which the Event Loop processes.

3. What is the Call Stack?

Interview Answer:

The Call Stack is where JavaScript executes synchronous code. It follows a Last-In-First-Out structure, meaning functions are added when called and removed when completed. Async callbacks can only run when the stack is empty.

4. What Are Task Queues?

Interview Answer:

Task queues store callbacks from asynchronous operations that are ready to run. The Event Loop pushes them to the Call Stack when it is free.

There are two main types:

- **Microtask Queue (High Priority) → Promises, queueMicrotask**
- **Macrotask Queue (Low Priority) → setTimeout, setInterval, I/O**

5. Difference Between Microtasks and Macrotasks?

Interview Answer (very commonly asked):

Microtasks have higher priority than macrotasks. After executing synchronous code, JavaScript completes all microtasks first, and only then executes one macrotask before repeating the cycle.

6. Why Does `setTimeout(fn, 0)` Not Run Immediately?

Interview Answer:

Because `setTimeout` places the callback into the macrotask queue. It must wait for the current execution and all microtasks to finish before the Event Loop can execute it, so it is never truly immediate.

7. How Does `async/await` Work Internally?

Interview Answer:

`async/await` is syntactic sugar over Promises. The `await` pauses execution of that function and schedules the remaining code as a microtask, allowing the Event Loop to continue processing other tasks.

8. What Happens If We Block the Event Loop?

Interview Answer:

If we run CPU-intensive synchronous code, the Event Loop cannot process queued tasks, causing UI freezes in browsers or blocking all requests in Node.js. That's why heavy computation should be moved to worker threads.

9. What is the Role of the Event Loop in Node.js?

Interview Answer:

In Node.js, the Event Loop coordinates non-blocking I/O using the libuv system. It delegates operations like file reading or networking to the OS or thread pool and processes their callbacks in phases, enabling Node.js to handle many concurrent requests efficiently.

10. Execution Order Rule You Should Always Remember?

Golden Rule (say this confidently):

JavaScript executes:

Synchronous Code → All Microtasks → One Macrotask → Repeat

11. Why Is the Event Loop Important for Scalability?

Interview Answer:

Because it allows JavaScript to handle thousands of operations without creating new threads for each task, making it lightweight and efficient for real-time applications and APIs.